

# Learning with ELK

*A Complete, Reorganized Guide to Elasticsearch, Logstash & Kibana*

Notes compiled: 31/07/2026 – 02/08/2026

# 1. Introduction to the ELK Stack

---

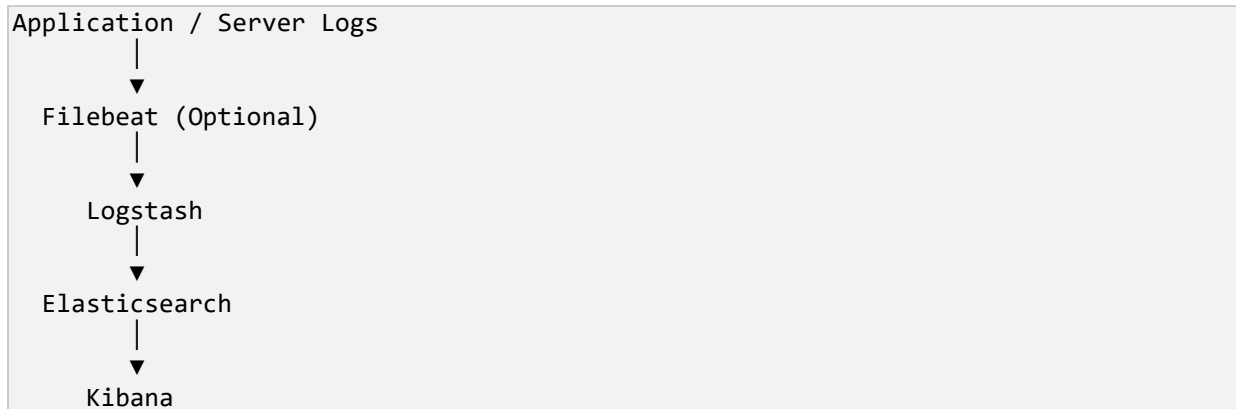
## What is the ELK Stack?

ELK Stack is a collection of open-source tools used to collect, process, store, search, analyze, and visualize logs generated by applications, servers, containers, and cloud infrastructure.

**ELK** = Elasticsearch + Logstash + Kibana

Elastic later introduced Beats, so the full stack is often called the Elastic Stack.

## High-Level Data Flow



## What Does "Distributed" Mean?

When we say Elasticsearch is a distributed search and analytics engine, it means:

- Data is stored on multiple servers (nodes).
- Each server holds part of the data.
- All servers work together as a single Elasticsearch cluster.
- If one server fails, others can continue serving requests (if replicas exist).

# 2. Elasticsearch Core Concepts

---

## What is Elasticsearch?

Elasticsearch is a distributed search and analytics engine where all processed logs are stored — think of it as a high-speed database for logs. Instead of using SQL like MySQL, Elasticsearch stores data as JSON documents.

```
{
  "timestamp": "2026-07-31T10:00:00",
  "level": "ERROR",
  "service": "payment-api",
  "message": "Database connection failed"
}
```

**What Elasticsearch does:** stores logs, searches millions of logs quickly, filters logs, aggregates data, performs analytics, and indexes documents.

## Index, Document, Field, Shard & Replica

### Index

An Index is a collection of related documents — similar to a database in MySQL or MongoDB. Example: logs-2026.07.31 stores all logs generated on 31 July 2026.

### Document

A Document is a single record stored inside an index, in JSON format:

```
{
  "user": "Anil",
  "status": "success",
  "ip": "192.168.1.10",
  "timestamp": "2026-07-31T10:20:00"
}
```

### Field

A Field is an individual key inside a document, e.g. "user", "status", "age".

### Shard

A Shard is a smaller piece of an index. Instead of storing all data on one server, Elasticsearch divides an index into multiple shards, each of which can live on a different node.

Example: a 300 GB index could be split into Shard 1 (100 GB), Shard 2 (100 GB), and Shard 3 (100 GB).

**Why use shards?** Faster search, parallel processing, easy scaling, and large-data handling. When you search for "ERROR", all shards search simultaneously and return results much faster.

### Replica

A Replica is a copy of a primary shard. If the primary server crashes, Elasticsearch automatically uses the replica.

## SQL vs Elasticsearch Terminology

| SQL      | Elasticsearch         |
|----------|-----------------------|
| Database | Index                 |
| Table    | (no exact equivalent) |
| Row      | Document              |
| Column   | Field                 |

## Quick Reference

| Concept  | Easy Meaning                    |
|----------|---------------------------------|
| Index    | Database                        |
| Document | One JSON record                 |
| Field    | One key/value inside a document |

| Concept        | Easy Meaning                                       |
|----------------|----------------------------------------------------|
| Shard          | One piece of an index                              |
| Replica        | Copy of a shard                                    |
| Cluster        | Group of Elasticsearch nodes                       |
| Node           | One Elasticsearch server                           |
| Distributed    | Data spread across multiple nodes working together |
| Scalable       | Add more nodes as data grows                       |
| Near Real-Time | New data becomes searchable in about 1 second      |

## 3. Cluster & Node Types

A Node is a single running Elasticsearch instance, e.g. each Docker container running Elasticsearch is one node.

### 1. Master Node

Controls the cluster. It does NOT store your documents. It manages cluster state, node joining/leaving, index creation/deletion, and shard/replica allocation.

```
node.roles:
  - master
```

### 2. Data Node

This is where your logs are actually stored. Responsibilities: store documents, search documents, index documents, and run aggregations.

```
node.roles:
  - data
```

### 3. Ingest Node

Responsible for processing documents before they are indexed, using ingest pipelines to transform, enrich, filter, or modify incoming data — reducing preprocessing work in applications or Logstash.

```
node.roles:
  - ingest
```

### 4. Coordinating Node

Does not store data. It only receives requests, sends them to data nodes, collects results, and returns the final response.

```
node.roles: [ ] # no roles assigned
```

### 5. Machine Learning Node

Runs anomaly detection, forecasting, and ML jobs.

```
node.roles:
  - ml
```

## Example Cluster Layout

| IP         | Role                  | Name           |
|------------|-----------------------|----------------|
| 172.20.0.2 | m (Master)            | es-master      |
| 172.20.0.3 | d (Data)              | es-data        |
| 172.20.0.4 | i (Ingest)            | es-ingest      |
| 172.20.0.5 | - (Coordinating-only) | es-coordinator |

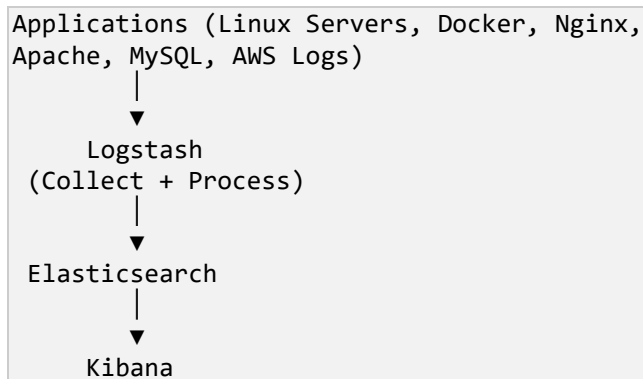
## 4. Beats

Beats are lightweight agents installed on servers to collect data and send it to Logstash or directly to Elasticsearch. Instead of Logstash reading log files from every server, Beats run on each machine and forward the data.

| Beat       | Purpose                 |
|------------|-------------------------|
| Filebeat   | Collect log files       |
| Metricbeat | Collect metrics         |
| Packetbeat | Collect network traffic |
| Winlogbeat | Windows Event Logs      |
| Auditbeat  | Linux audit logs        |
| Heartbeat  | Uptime monitoring       |

## 5. Logstash

Logstash is a server-side data processing pipeline that collects data from multiple sources, processes or transforms it, and sends it to Elasticsearch or other destinations.



### 1. Input Plugin

Collects data from different sources: log files, Docker logs, Jenkins logs, Linux syslogs, Beats (Filebeat, Metricbeat), Kafka, RabbitMQ, TCP/UDP, HTTP, MySQL.

### 2. Filter Plugin

Modifies or enriches the data: parses logs, removes unnecessary fields, renames fields, converts data types, adds new fields, extracts IP addresses/usernames.

| Plugin | Purpose                             |
|--------|-------------------------------------|
| grok   | Extracts structured data from text  |
| mutate | Renames, removes, or changes fields |

| Plugin | Purpose                                      |
|--------|----------------------------------------------|
| date   | Converts timestamps                          |
| geoip  | Adds location information from an IP address |
| json   | Parses JSON logs                             |

### 3. Output Plugin

Sends the processed data to a destination: Elasticsearch, File, Kafka, AWS S3, Email, or Stdout (console).

### Sample Logstash Configuration

```
input {
  file {
    path => "/var/log/app.log"
    start_position => "beginning"
  }
}

filter {
  grok {
    match => { "message" => "%{TIMESTAMP_ISO8601:time} %{LOGLEVEL:level}
%{GREEDYDATA:msg}" }
  }
}

output {
  elasticsearch {
    hosts => ["localhost:9200"]
    index => "application-logs"
  }
}
```

## 6. Kibana

Kibana is the visualization and analytics tool of the Elastic Stack. It reads data from Elasticsearch and allows you to search, analyze, and visualize data through dashboards, charts, and graphs — including creating alerts and monitoring applications in real time.

### Key Features

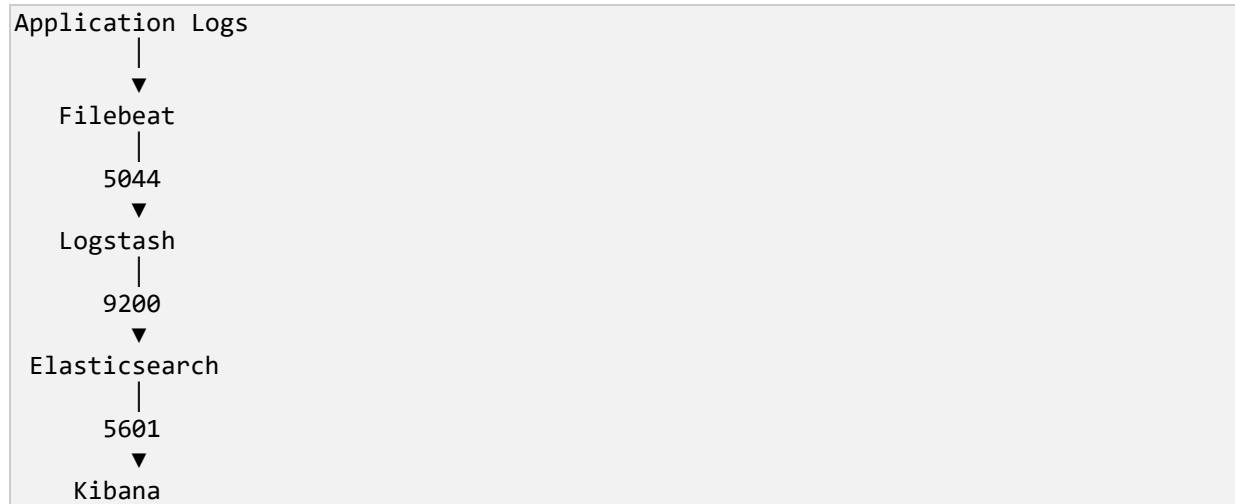
- Discover — search logs.
- Dashboard — combine multiple visualizations.
- Visualize — create pie charts, bar charts, line graphs, heat maps, and data tables.

## 7. Default Ports

| Port | Used By                                                           |
|------|-------------------------------------------------------------------|
| 9200 | Elasticsearch client REST API (HTTP) — curl http://localhost:9200 |
| 9300 | Elasticsearch internal node-to-node communication                 |
| 5601 | Kibana — http://localhost:5601                                    |

| Port | Used By                     |
|------|-----------------------------|
| 5044 | Logstash (used by Filebeat) |
| 5000 | Logstash TCP/UDP log input  |
| 8080 | Logstash HTTP input plugin  |

## End-to-End Flow with Ports



## 8. CRUD Operations

### 1. Create an Index

```
POST employees
```

### 2. Create a Document (auto-generated ID)

```
POST employees/_doc
{
  "name": "Anil",
  "department": "DevOps",
  "salary": 50000
}
```

### 3. Read a Document by ID

```
GET employees/_doc/1
```

### 4. Read All Documents

```
GET employees/_search
```

### 5. Read with a Query

```
GET employees/_search
{
  "query": {
    "match": { "department": "DevOps" }
  }
}
```

```
}  
}
```

## 6. Update a Document

```
POST employees/_update/1  
{  
  "doc": { "salary": 60000 }  
}
```

## 7. Delete a Document

```
DELETE employees/_doc/1
```

## 8. Upsert (Update + Insert)

If the document exists, update it; if it does not exist, insert it. This avoids checking whether the document already exists.

```
POST employees/_update/1  
{  
  "doc": { "salary": 70000 },  
  "doc_as_upsert": true  
}
```

### *Explicit upsert with a script*

```
POST employees/_update/10  
{  
  "script": {  
    "source": "ctx._source.salary += params.amount",  
    "params": { "amount": 5000 }  
  },  
  "upsert": { "name": "Anil", "salary": 5000 }  
}
```

## 9. Update Multiple Fields via Script

```
{  
  "script": {  
    "source": ""  
    ctx._source.salary = params.salary;  
    ctx._source.department = params.department;  
    "",  
    "params": { "salary": 90000, "department": "Cloud" }  
  }  
}
```

## 10. Remove a Field

```
{  
  "script": { "source": "ctx._source.remove('salary')" }  
}
```

## 11. Update By Query



Updates multiple documents matching a query.

```
POST employees/_update_by_query
{
  "script": { "source": "ctx._source.salary += 5000" },
  "query": { "match": { "department": "DevOps" } }
}
```

## 12. Delete By Query

```
POST employees/_delete_by_query
{
  "query": { "match": { "department": "Testing" } }
}
```

## 9. Routing

---

Routing is the process Elasticsearch uses to determine which primary shard a document should be stored in, and which shard should be queried when searching for that document. Without routing, Elasticsearch would have to search every shard, which is slower.

### Default Routing

By default, Elasticsearch uses the document's `_id` as the routing value.

### Routing Formula

```
Shard Number = hash(routing) % Number_of_Primary_Shards
```

### Custom Routing

```
PUT employees/_doc/1?routing=devops
```

### Advantages

- Faster searches.
- Fewer shards queried.
- Reduced CPU usage.
- Better performance for tenant-based applications.
- Useful for multi-tenant architectures.

### Disadvantages

If many documents use the same routing value (e.g. "DevOps"), millions of documents may end up in one shard. This creates a hot shard, causing uneven storage and workload distribution.

## 10. Sequence Number, Primary Term & Checkpoints

---

### `_seq_no` (Sequence Number)

Every write operation in Elasticsearch (create, update, delete) is assigned a unique sequence number by the primary shard — think of it like a ticket number. Each operation increments the sequence number.

## **\_primary\_term**

The primary term identifies the current primary shard's generation. Whenever a new primary shard is elected, the primary term increases.

```
Primary Shard Dies
↓
Replica becomes Primary
↓
Primary Term = 2 (was 1)
```

Why does it exist? If the primary crashes and a replica becomes the new primary, an old primary that comes back with stale data must have its outdated writes rejected. The primary term distinguishes the new primary from the old one.

## **\_seq\_no vs \_primary\_term**

| <b>_seq_no</b>         | <b>_primary_term</b>                        |
|------------------------|---------------------------------------------|
| Changes on every write | Changes only when the primary shard changes |
| Tracks operation order | Tracks primary shard generation             |
| Used for concurrency   | Prevents stale primary writes               |

## **Global Checkpoint**

The highest sequence number that has been successfully processed on all in-sync shard copies.

## **Local Checkpoint**

The highest sequence number that an individual shard has successfully processed without gaps.

# **11. Optimistic Concurrency Control (OCC)**

OCC is a mechanism used by Elasticsearch to prevent multiple users or applications from accidentally overwriting each other's changes. It assumes conflicts are rare, but when they happen, Elasticsearch detects them and rejects the conflicting update.

| <b>Question</b>              | <b>Answer</b>                                                             |
|------------------------------|---------------------------------------------------------------------------|
| What is OCC?                 | Prevents stale updates in concurrent writes.                              |
| Which fields are used?       | _seq_no and _primary_term.                                                |
| What happens on mismatch?    | Elasticsearch returns 409 Conflict.                                       |
| Why not use locks?           | OCC gives better performance since conflicts are expected to be uncommon. |
| What problem does OCC solve? | Lost updates caused by concurrent modifications.                          |

# **12. Bulk API**

The Bulk API allows you to perform multiple operations (create, index, update, delete) in a single HTTP request instead of sending one request per document. This significantly reduces network overhead and improves indexing performance.

```
POST _bulk
{ "index": { "_index": "employees", "_id": "1" } }
{ "name": "Anil", "salary": 50000 }

{ "update": { "_index": "employees", "_id": "1" } }
{ "doc": { "salary": 60000 } }

{ "delete": { "_index": "employees", "_id": "2" } }

{ "create": { "_index": "employees", "_id": "3" } }
{ "name": "Rahul", "salary": 70000 }
```

## 13. Analyzer

---

An Analyzer is a component that converts raw text into searchable tokens during indexing, and also analyzes the search query using the same process.

- Character Filter — modifies the text before tokenization.
- Tokenizer — breaks text into tokens (words).
- Token Filter — modifies the resulting tokens.

## 14. Inverted Index

---

An Inverted Index is a data structure that maps terms (words) to the documents in which they appear, enabling extremely fast full-text searches.

### Why "Inverted"?

A traditional (forward) index stores data as Document → Words. Searching for a term may require scanning many rows unless a suitable index exists (a full table scan,  $O(n)$ ).

| Document ID | Content              |
|-------------|----------------------|
| 1           | DevOps Engineer      |
| 2           | Java Developer       |
| 3           | DevOps Administrator |
| 4           | Python Developer     |

An inverted index instead stores Word → Documents:

| Term          | Documents |
|---------------|-----------|
| devops        | 1, 3      |
| engineer      | 1         |
| java          | 2         |
| developer     | 2, 4      |
| administrator | 3         |
| python        | 4         |

Now searching "devops" directly finds Doc1 and Doc3 — no need to scan every document. Time complexity is roughly  $O(\log n)$  instead of  $O(n)$ .

## 15. Doc Values

---

Doc Values are an on-disk data structure Elasticsearch uses to efficiently retrieve field values for sorting, aggregations, and scripting. Unlike the inverted index (optimized for searching), doc values are optimized for accessing values by document.

### Why Are They Needed?

An inverted index stores values like "60000 → Doc3", which is great for searching salary:60000 but inefficient for repeatedly retrieving each document's salary during sorting. Doc values instead store the data as Doc1 → 50000, Doc2 → 70000, Doc3 → 60000, so Elasticsearch can quickly read each document's value.

### Inverted Index vs Doc Values

| Feature          | Inverted Index   | Doc Values                     |
|------------------|------------------|--------------------------------|
| Structure        | Term → Documents | Document → Value               |
| Used For         | Search, filters  | Sorting, aggregations, scripts |
| Full-text search | Yes              | No                             |
| Sort             | No               | Yes                            |
| Aggregation      | No               | Yes                            |
| Phrase search    | Yes              | No                             |

### Which Fields Have Doc Values by Default?

| Field Type | Doc Values |
|------------|------------|
| keyword    | Yes        |
| date       | Yes        |
| integer    | Yes        |
| long       | Yes        |
| float      | Yes        |
| boolean    | Yes        |
| ip         | Yes        |
| geo_point  | Yes        |
| text       | No         |

### Disabling Doc Values

```
"salary": {
  "type": "integer",
  "doc_values": false
}
```

This saves disk space but means the field cannot be used efficiently for sorting, aggregations, or most scripts.

## 16. Elasticsearch Mapping

---

A mapping defines how Elasticsearch stores and indexes each field in a document — similar to a schema in a relational database.

| SQL Database | Elasticsearch |
|--------------|---------------|
| Database     | Cluster       |
| Table        | Index         |
| Row          | Document      |
| Column       | Field         |
| Schema       | Mapping       |

## Example Mapping

```
PUT employees
{
  "mappings": {
    "properties": {
      "id": { "type": "integer" },
      "name": { "type": "text" },
      "salary": { "type": "integer" },
      "joining_date": { "type": "date" }
    }
  }
}
```

Without mapping, Elasticsearch would not know whether a value like 100 is a number or text, whether a date string should be treated as a date, or whether a field like "DevOps Engineer" should support full-text search.

## 1. Dynamic Mapping

By default, Elasticsearch uses dynamic mapping — when you index a document with a new field, Elasticsearch automatically detects its data type.

**Advantages:** very easy to use, great for development, good for quickly ingesting logs, no schema definition required.

```
POST employees/_doc/1
{
  "name": "Anil",
  "age": 25,
  "salary": 50000,
  "active": true
}
```

Elasticsearch automatically creates a mapping similar to:

```
{
  "properties": {
    "name": { "type": "text" },
    "age": { "type": "long" },
    "salary": { "type": "long" },
    "active": { "type": "boolean" }
  }
}
```

**Disadvantages:** a quoted value like "50000" may be detected as a string; a later numeric insert then causes a mapping conflict. Dynamic mapping can also create unwanted fields due to typos.

## 2. Explicit Mapping

In explicit mapping, you define the mapping before indexing any documents, and Elasticsearch must follow those data types.

```
PUT employees
{
  "mappings": {
    "properties": {
      "name": { "type": "text" },
      "salary": { "type": "integer" },
      "joining_date": { "type": "date" },
      "department": { "type": "keyword" }
    }
  }
}
```

Invalid data (e.g. "salary": "abc") is rejected with an error such as "Failed to parse field [salary] as integer" because the mapping is enforced.

## Dynamic vs Explicit Mapping

| Feature             | Dynamic Mapping            | Explicit Mapping        |
|---------------------|----------------------------|-------------------------|
| Schema creation     | Automatic                  | Manual                  |
| Easy to start       | Yes                        | No                      |
| Control             | Low                        | High                    |
| Risk of wrong types | High                       | Very low                |
| Production use      | Not recommended alone      | Recommended             |
| Best for            | Development, log ingestion | Production applications |

## Common Field Types

| Type      | Example              |
|-----------|----------------------|
| text      | "DevOps Engineer"    |
| keyword   | "EMP001"             |
| integer   | 100                  |
| long      | 9999999999           |
| float     | 10.5                 |
| double    | 20.75                |
| boolean   | true                 |
| date      | "2026-08-03"         |
| ip        | "192.168.1.10"       |
| geo_point | latitude & longitude |

## The "dynamic" Setting

The dynamic setting controls what happens when a document contains new fields not defined in the mapping. There are four possible values:

- dynamic: true (default) — new fields are automatically added to the mapping.

- `dynamic: false` — unknown fields are stored in `_source` but not added to the mapping, and cannot be searched or aggregated.
- `dynamic: strict` — unknown fields are rejected completely.
- `dynamic: runtime` — unknown fields become runtime fields instead of being indexed.

## Mapping API

Think of it like SQL's CREATE TABLE, ALTER TABLE, and DESCRIBE TABLE.

### *Get Mapping*

```
GET employees/_mapping
```

### *Get Mapping of All Indices*

```
GET _mapping
```

### *Add a New Field to an Existing Mapping*

```
PUT employees/_mapping
{
  "properties": {
    "email": { "type": "keyword" }
  }
}
```

### *Can You Change a Field's Type?*

No. Attempting to change salary from integer to text returns an error: "mapper [salary] cannot be changed from type [integer] to [text]". Elasticsearch does not allow changing the data type of an existing field.

### *Check a Specific Field's Mapping*

```
GET employees/_mapping/field/salary
```

### *Deleting a Mapping / Field*

There is no API to delete a mapping or remove a field from an existing index. To remove a field: create a new index with the desired mapping, reindex the data, then delete the old index if it's no longer needed.

## 17. Reindex API

---

The Reindex API copies documents from one index to another. It is commonly used to change a field's mapping, rename an index, migrate data, change analyzers/settings, or filter/transform documents while copying.

### Reindex Selected Documents

```
POST _reindex
{
  "source": {
    "index": "employees",
    "query": { "range": { "salary": { "gt": 50000 } } }
  }
```

```
    },  
    "dest": { "index": "high_salary" }  
  }  
}
```

## Reindex While Transforming Data

```
POST _reindex  
{  
  "source": { "index": "employees" },  
  "dest": { "index": "employees_new" },  
  "script": { "source": "ctx._source.bonus = 10000" }  
}
```

## 18. Coercion

Coercion is the process where Elasticsearch automatically converts a value into the expected data type when possible — if the value can be safely converted, Elasticsearch converts it instead of throwing an error.

### Example: Float to Integer

With coercion enabled, 50000.99 becomes 50000 — the fractional part is discarded when converting to an integer.

### Example: Invalid Value

Inserting "salary": "abc" fails with "Cannot convert 'abc' into integer" — coercion only works when the conversion is valid.

### Supported Conversions

| Input   | Field Type | Result          |
|---------|------------|-----------------|
| "25"    | integer    | 25 (success)    |
| "50000" | long       | 50000 (success) |
| 100.99  | integer    | 100 (success)   |
| "true"  | boolean    | true (success)  |
| "false" | boolean    | false (success) |
| "abc"   | integer    | Error           |

### Disabling Coercion

#### *For a single field*

```
PUT employees  
{  
  "mappings": {  
    "properties": {  
      "salary": { "type": "integer", "coerce": false }  
    }  
  }  
}
```



With coercion disabled, inserting "salary": "50000" now fails with "failed to parse field [salary]".

#### *For an entire index*

```
PUT employees
{
  "settings": {
    "index.mapping.coerce": false
  },
  "mappings": {
    "properties": {
      "age": { "type": "integer" }
    }
  }
}
```

## 19. Index Templates

---

An Index Template is a reusable blueprint that automatically applies settings, mappings, and aliases to new indices whose names match a specified pattern.

### Why Do We Need Them?

Imagine daily log indices such as logs-2026.08.01, logs-2026.08.02, logs-2026.08.03... Without a template you would have to configure shards, replicas, mappings, and aliases manually for every new index — repetitive and error-prone. With a template, Elasticsearch applies the configuration automatically whenever a matching index is created.

### Create an Index Template

```
PUT _index_template/logs_template
{
  "index_patterns": ["logs-*"],
  "template": {
    "settings": {
      "number_of_shards": 1,
      "number_of_replicas": 1
    },
    "mappings": {
      "properties": {
        "timestamp": { "type": "date" },
        "level": { "type": "keyword" },
        "message": { "type": "text" }
      }
    }
  }
}
```

### Create a New Index (Template Auto-Applies)

```
PUT logs-2026.08.03
```

## 20. Elastic Common Schema (ECS)

---

ECS is a standard field naming convention created by Elasticsearch so that logs, metrics, traces, and security events from different sources all use the same field names and structure — think of it as a common language for data.

### Why Is ECS Needed?

Different applications often describe the same concept with different field names, e.g. "username" vs "user\_name" vs "user", or "ip" vs "client\_ip" vs "source\_ip". This makes searching across sources difficult.

### ECS Standard Example

```
{
  "user": { "name": "anil" },
  "source": { "ip": "192.168.1.10" }
}
```

Every application sending data in this structure becomes searchable and comparable in the same way.

## 21. Aliases

---

An Alias is a virtual name that points to one or more Elasticsearch indices — a shortcut or logical name for an index. Applications use the alias instead of the actual index name.

### Zero-Downtime Migration

Suppose alias "employees" points to "employees\_v1". Create a new index, reindex the data into it, then switch the alias:

```
PUT employees_v2

POST _reindex
{
  "source": { "index": "employees_v1" },
  "dest": { "index": "employees_v2" }
}

POST _aliases
{
  "actions": [
    { "remove": { "index": "employees_v1", "alias": "employees" } },
    { "add": { "index": "employees_v2", "alias": "employees" } }
  ]
}
```

The application continues to use GET employees/\_search without any changes, now pointing at employees\_v2.

### Filtered Alias

A filtered alias returns only documents that match a predefined filter.

```
POST _aliases
{
  "actions": [
    {
      "add": {
        "index": "employees",
        "alias": "it_employees",
        "filter": { "term": { "department": "IT" } }
      }
    }
  ]
}

GET it_employees/_search
```

## 22. Cross-Cluster Search vs Cross-Cluster Replication

| Cross-Cluster Search | Cross-Cluster Replication  |
|----------------------|----------------------------|
| Searches remote data | Copies remote data         |
| No duplication       | Data duplicated            |
| Read-only            | Replication                |
| Used for analytics   | Used for disaster recovery |

## 23. Full-Text Queries

Full-text queries search analyzed text fields. Before searching, Elasticsearch breaks the text into tokens using an analyzer, making searches flexible and relevant.

### How Analysis Works

A document with the title "Learning Elasticsearch with Docker" is stored as tokens: learning, elasticsearch, with, docker. A search for "Learn Elasticsearch" is also analyzed into tokens (learn, elasticsearch) and compared against the stored tokens rather than the original sentence.

### 1. Match Query

The most commonly used full-text query — it analyzes the search text before searching.

```
GET books/_search
{
  "query": {
    "match": { "title": "elasticsearch docker" }
  }
}
```

By default the match query uses an OR operator between tokens. You can force AND instead:

```
GET books/_search
{
  "query": {
```

```
    "match": {
      "title": { "query": "java docker", "operator": "and" }
    }
  }
}
```

## 2. Match Phrase Query

Searches for an exact phrase — word order matters.

```
GET books/_search
{
  "query": {
    "match_phrase": { "title": "Elasticsearch with Docker" }
  }
}
```

Searching "docker elasticsearch" would return no result, because the order changed.

## 3. Match Phrase Prefix Query

Used for autocomplete / search-as-you-type.

```
GET books/_search
{
  "query": {
    "match_phrase_prefix": { "title": "Learning Ela" }
  }
}
```

Useful for search suggestions, autocomplete, and live search.

## 4. Multi Match Query

Searches multiple fields at once.

```
GET books/_search
{
  "query": {
    "multi_match": {
      "query": "docker",
      "fields": ["title", "description"]
    }
  }
}
```

- `best_fields` (default) — the highest-scoring field wins.
- `most_fields` — combines scores from multiple fields; useful when several analyzed versions exist.
- `cross_fields` — treats several fields (e.g. `first_name` and `last_name`) as one combined field.

## 24. Disjunction Max Query (`dis_max`)

---

dis\_max is used when you want to search multiple queries but only the best matching query should determine the document's score. Unlike a bool query with "should" (which adds up scores), dis\_max uses the highest score, optionally adding a small bonus via tie\_breaker.

```
GET products/_search
{
  "query": {
    "dis_max": {
      "tie_breaker": 0.3,
      "queries": [
        { "match": { "title": "iphone" } },
        { "match": { "description": "iphone" } }
      ]
    }
  }
}
```

For example, if the title score is 5 and the description score is 3: without a tie\_breaker the final score is simply 5; with tie\_breaker = 0.3 the final score is  $5 + (3 \times 0.3) = 5.9$  — the best score is used in full while the other matches contribute only a fraction.

dis\_max is useful when a match in one field (e.g. title) should matter more than the same match in another field (e.g. description), helping rank documents by their best field match.

## 25. IDs Query

---

A term-level query used to retrieve documents by their known \_id values.

```
GET employees/_search
{
  "query": {
    "ids": { "values": ["101", "102", "103"] }
  }
}
```

## 26. Bool Query — Combining Multiple Queries

---

The bool query combines one or more query clauses using logical operators.

```
GET employees/_search
{
  "query": {
    "bool": {
      "must": [],
      "should": [],
      "filter": [],
      "must_not": []
    }
  }
}
```

| Clause | SQL Equivalent | Affects Score? | Purpose                   |
|--------|----------------|----------------|---------------------------|
| must   | AND            | Yes            | All conditions must match |

| Clause   | SQL Equivalent | Affects Score? | Purpose                                       |
|----------|----------------|----------------|-----------------------------------------------|
| should   | OR             | Yes            | At least one should match (context-dependent) |
| filter   | AND            | No             | Exact filtering without affecting score       |
| must_not | NOT            | No             | Exclude matching documents                    |

## minimum\_should\_match

Require a minimum number of "should" clauses to match.

```
GET employees/_search
{
  "query": {
    "bool": {
      "should": [
        { "match": { "skills": "Docker" } },
        { "match": { "skills": "Kubernetes" } },
        { "match": { "skills": "AWS" } }
      ],
      "minimum_should_match": 2
    }
  }
}
```

## Filter Clause

Filtering does not affect relevance scoring and is cacheable, making it faster for exact matches.

```
GET employees/_search
{
  "query": {
    "bool": {
      "filter": [ { "term": { "status": "ACTIVE" } } ]
    }
  }
}
```

## Must Not Clause

Excludes documents — equivalent to SQL's WHERE status != 'INACTIVE'.

```
GET employees/_search
{
  "query": {
    "bool": {
      "must_not": [ { "term": { "status": "INACTIVE" } } ]
    }
  }
}
```

## 27. Term Query

---

A term-level query that searches for an exact value in a field, without analyzing the search text — it looks for the exact value stored in the inverted index. It is mainly used on keyword, numeric, date, and boolean fields, and is not recommended for text fields because text fields are analyzed.

```
GET employees/_search
{
  "query": {
    "term": { "status": { "value": "ACTIVE" } }
  }
}
```

## Term Query vs Match Query

| Feature        | Term Query                      | Match Query                      |
|----------------|---------------------------------|----------------------------------|
| Analyzer       | No                              | Yes                              |
| Exact match    | Yes                             | No                               |
| Used for       | keyword, numeric, date, boolean | text                             |
| Case sensitive | Usually yes (keyword)           | Usually no (depends on analyzer) |
| Tokenization   | No                              | Yes                              |

## term vs terms

term matches a single exact value; terms matches any one of multiple exact values. Use status.keyword instead of status when you need an exact match on a field that has both a text mapping and a keyword subfield.

## 28. Range Query

Finds documents whose field value is greater than, less than, greater than or equal to, or less than or equal to a specified value.

```
GET employees/_search
{
  "query": {
    "range": {
      "salary": { "gte": 50000, "lte": 100000 }
    }
  }
}
```

| Operator | Meaning                    |
|----------|----------------------------|
| gt       | Greater than (>)           |
| gte      | Greater than or equal (>=) |
| lt       | Less than (<)              |
| lte      | Less than or equal (<=)    |

## 29. Exists Query

Returns documents where a specified field contains an indexed value. Commonly used to find documents with missing data, filter documents with required fields, or check whether a field is present.

```
GET employees/_search
{
  "query": {
    "exists": { "field": "email" }
  }
}
```

## 30. Wildcard Query

---

A term-level query that searches for documents matching a pattern using wildcard characters — useful when you know only part of a value such as a username, email, filename, or product code. Works best on keyword fields and is not recommended for text fields.

| Symbol | Meaning                         |
|--------|---------------------------------|
| *      | Matches zero or more characters |
| ?      | Matches exactly one character   |

```
GET employees/_search
{
  "query": {
    "wildcard": { "name.keyword": { "value": "Ani*" } }
  }
}
```

## 31. Prefix Query

---

Returns documents whose field values begin with the specified prefix. Works best on keyword, numeric strings, SKU/product codes, and email usernames — not recommended for text fields.

```
GET employees/_search
{
  "query": {
    "prefix": { "name.keyword": { "value": "Ani" } }
  }
}
```

## 32. Nested Query

---

Used to search nested objects while preserving the relationship between fields inside the same object — needed when a document contains arrays of objects.

```
GET employees/_search
{
  "query": {
    "nested": {
      "path": "skills",
      "query": {
        "bool": {
          "must": [
            { "term": { "skills.technology": "Docker" } },
            { "term": { "skills.experience": 3 } }
          ]
        }
      }
    }
  }
}
```



```
}
  }
}
}
```

## object vs nested

| object                    | nested                                                  |
|---------------------------|---------------------------------------------------------|
| Flattens data             | Stores each array element as a separate hidden document |
| Relationships can be lost | Relationships are preserved                             |

The path parameter tells Elasticsearch which nested field to search. Any valid query (bool, match, term, range, etc.) can be placed inside a nested query's query section. The biggest disadvantage of nested fields is that they increase index size and query complexity, since each nested object is indexed as a separate hidden document.

## 33. Query Types — Quick Revision

| Query    | Purpose                                                 |
|----------|---------------------------------------------------------|
| match    | Full-text search                                        |
| term     | Exact value                                             |
| range    | Numeric/date ranges                                     |
| exists   | Field existence                                         |
| prefix   | Starts with a value                                     |
| wildcard | Pattern matching                                        |
| regexp   | Regular expression matching                             |
| bool     | Combine multiple queries                                |
| nested   | Search arrays of objects while preserving relationships |

## 34. Joins in Elasticsearch

In Elasticsearch, joins are not like SQL joins. Since Elasticsearch is a distributed search engine, it avoids expensive runtime joins across documents — documents are stored independently across multiple shards, and performing joins across shards would be slow and expensive. Instead, Elasticsearch recommends several alternatives:

- Denormalization (most common)
- Nested Objects
- Parent-Child Relationship (join field)
- Terms Lookup
- Application-side joins
- Enrich Processor (during indexing)

### 1. Denormalization (Recommended)

Store all required data in a single document.

## 2. Nested Documents

Used when one document contains an array of related objects (see the Nested Query section above).

## 3. Parent-Child Join (Join Field)

Use when the parent and child are separate documents, e.g. Employee → Tasks.

```
PUT company
{
  "mappings": {
    "properties": {
      "relation": {
        "type": "join",
        "relations": { "employee": "task" }
      }
    }
  }
}

PUT company/_doc/1
{ "name": "Anil", "relation": "employee" }

PUT company/_doc/2?routing=1
{
  "task": "Deploy Kubernetes",
  "relation": { "name": "task", "parent": "1" }
}
```

### *Search Parent Having a Matching Child*

```
GET company/_search
{
  "query": {
    "has_child": {
      "type": "task",
      "query": { "match": { "task": "Kubernetes" } }
    }
  }
}
```

### *Search Child Having a Matching Parent*

```
GET company/_search
{
  "query": {
    "has_parent": {
      "parent_type": "employee",
      "query": { "match": { "name": "Anil" } }
    }
  }
}
```

**Important rules:** parent and child must be in the same index; child documents require the same routing value as the parent; this approach is slower than denormalization.

## 4. Terms Lookup Query

Lets Elasticsearch fetch a list of values from one document and use those values in a terms query against another index — similar to a SQL sub-query.

```
{
  "query": {
    "terms": {
      "product_id": {
        "index": "users",
        "id": "1",
        "path": "favorite_products"
      }
    }
  }
}
```

**Advantages:** no application code needed to fetch the lookup values; keeps queries concise.

**Limitations:** only works with a single document identified by its ID; the field at path should hold exact-match values (typically keyword or numeric); it is not a replacement for relational joins across many documents.

## 5. Application-Side Join

Your application (Java, Spring Boot, Node.js, Python, .NET, etc.) queries two indices separately and merges the results itself — the most common approach in production, since Elasticsearch is optimized for searching, not relational joins.

```
{
  "customer": { "id": 100, "name": "Anil", "city": "Delhi" },
  "orders": [
    { "order_id": 5001, "product": "Laptop" },
    { "order_id": 5002, "product": "Mouse" }
  ]
}
```

Real-world example: for a customer profile page, fetch the customer from the customers index, fetch their orders from the orders index using the customer's ID, merge the results in the backend, and return a single JSON response to the frontend. This scales well, keeps Elasticsearch queries simple, and is easy to maintain — though it requires two or more queries and merging logic in the application.

## 6. Enrich Processor

An ingest pipeline processor that looks up additional information from another index while a document is being indexed — a lookup that happens once during indexing rather than on every search, so the extra fields are stored permanently on the document.

**Advantages:** very fast and scalable; works across different indexes; simple to implement; no special join mappings required; easy to customize the final response.

**Disadvantages:** requires two or more Elasticsearch queries to set up; more application logic than a single SQL JOIN.

## 35. Aggregations

---

Aggregations in Elasticsearch are similar to GROUP BY and aggregate functions (COUNT, SUM, AVG, etc.) in SQL — they summarize, analyze, and calculate statistics on indexed data instead of returning individual documents.

### Types of Aggregations

- Metrics Aggregations
- Bucket Aggregations
- Pipeline Aggregations

#### 1. Metrics Aggregations

Calculate values such as count, sum, average, minimum, and maximum.

##### *value\_count*

```
GET products/_search
{
  "size": 0,
  "aggs": {
    "total_products": { "value_count": { "field": "price" } }
  }
}
```

##### *sum*

```
GET products/_search
{
  "size": 0,
  "aggs": {
    "total_price": { "sum": { "field": "price" } }
  }
}
```

##### *avg*

```
GET products/_search
{
  "size": 0,
  "aggs": {
    "average_price": { "avg": { "field": "price" } }
  }
}
```

##### *stats (multi-value)*

```
GET products/_search
{
  "size": 0,
  "aggs": {
    "price_stats": { "stats": { "field": "price" } }
  }
}
```

```
// Example result: { "count": 3, "min": 5000, "max": 50000, "avg": 28333, "sum": 85000 }
```

### ***cardinality***

Approximate count of unique values.

```
GET products/_search
{
  "size": 0,
  "aggs": {
    "unique_categories": { "cardinality": { "field": "category.keyword" } }
  }
}
```

## **2. Bucket Aggregations**

Group documents into buckets — equivalent to SQL's SELECT category, COUNT(\*) FROM products GROUP BY category.

### ***Terms Aggregation***

```
GET products/_search
{
  "size": 0,
  "aggs": {
    "categories": { "terms": { "field": "category.keyword" } }
  }
}
// Output: Electronics: 2, Furniture: 1
```

### ***Range Aggregation***

```
GET products/_search
{
  "size": 0,
  "aggs": {
    "price_ranges": {
      "range": {
        "field": "price",
        "ranges": [
          { "to": 10000 },
          { "from": 10000, "to": 40000 },
          { "from": 40000 }
        ]
      }
    }
  }
}
```

### ***Date Histogram***

```
GET sales/_search
{
  "size": 0,
  "aggs": {
    "sales_per_month": {
```

```

    "date_histogram": { "field": "date", "calendar_interval": "month" }
  }
}

```

### **Histogram**

Groups numeric values into fixed-size ranges (buckets) — e.g. products grouped into price bands of ₹0–9,999, ₹10,000–19,999, and so on, instead of listing every individual price.

```

GET products/_search
{
  "size": 0,
  "aggs": {
    "price_histogram": { "histogram": { "field": "price", "interval": 10000 } }
  }
}

```

### **Filter Aggregation**

Runs an aggregation only on the documents matching a specified filter.

```

GET products/_search
{
  "size": 0,
  "aggs": {
    "electronics": { "filter": { "term": { "category.keyword": "Electronics" } } }
  }
}

```

### **Filters Aggregation (Multiple Filters)**

Lets you define multiple filters in a single query — like asking several questions in one request.

```

GET products/_search
{
  "size": 0,
  "aggs": {
    "categories": {
      "filters": {
        "filters": {
          "electronics": { "term": { "category.keyword": "Electronics" } },
          "furniture": { "term": { "category.keyword": "Furniture" } }
        }
      }
    }
  }
}

```

### **Nested Aggregation**

If a field is mapped as nested, you must use a nested aggregation to calculate metrics such as avg, sum, min, max, or terms on the nested objects.

```

GET students/_search
{
  "size": 0,

```

```

"aggs": {
  "subjects": {
    "nested": { "path": "subjects" },
    "aggs": {
      "avg_marks": { "avg": { "field": "subjects.marks" } }
    }
  }
}

```

## SQL vs Elasticsearch Aggregations

| SQL            | Elasticsearch                          |
|----------------|----------------------------------------|
| GROUP BY       | terms aggregation                      |
| COUNT()        | value_count                            |
| SUM()          | sum                                    |
| AVG()          | avg                                    |
| MIN()          | min                                    |
| MAX()          | max                                    |
| DISTINCT COUNT | cardinality                            |
| HAVING         | bucket_selector (pipeline aggregation) |
| Date grouping  | date_histogram                         |

## 36. Fuzziness

Fuzziness allows Elasticsearch to find documents even when the search term contains typos, spelling mistakes, or minor differences. It is based on Levenshtein Edit Distance, which measures how many single-character edits are needed to transform one word into another. Allowed edits are: insert a character, delete a character, replace a character, or transpose (swap) adjacent characters.

### Manual Fuzziness

```

GET students/_search
{
  "query": {
    "match": {
      "name": { "query": "Anil", "fuzziness": 1 }
    }
  }
}

```

### AUTO Fuzziness

Instead of specifying 1 or 2 manually, Elasticsearch can decide the edit distance automatically based on term length:

```

GET students/_search
{
  "query": {
    "match": {
      "name": { "query": "Elasticsarch", "fuzziness": "AUTO" }
    }
  }
}

```

```
}  
}
```

| Word Length | Edit Distance |
|-------------|---------------|
| 0-2         | 0             |
| 3-5         | 1             |
| 6+          | 2             |

## Fuzzy Query

A dedicated fuzzy query gives more control than a match query with fuzziness.

```
GET students/_search  
{  
  "query": {  
    "fuzzy": {  
      "name": { "value": "Anll", "fuzziness": 1 }  
    }  
  }  
}
```